Izdelava urnikov

Njec irovnik

14. 10. 2019

Povzetek

V lanku so predstavljeni nekateri postopki za avtomatsko izdelavo urnikov. Opisani so potrebni vhodni podatki, natete so omejitve, ki jih je potrebno upotevati pri izdelavi urnikov, predstavljena je tudi metoda merjenja kakovosti izdelanega urnika.

1 Program za izdelavo urnika

Spominjam se osnovnoolskih in srednjeolskih dni, ko smo vsi z nestrpnostjo priakovali zaetek novega olskega leta, ki pa je nemalokrat prinesel zelo hladen tu ob pogledu na natrpan urnik. Zakaj moramo v ponedeljek tako zgodaj vstati, ko pa se teden e ni niti prav zael? Zakaj nam vsaj v petek niso malo popustili? Kdo je sploh lahko izdelal taken urnik?

To so vpraanja, na katera seveda nikoli nismo dobili odgovora, eprav je le-ta na zadnje vpraanje ve kot oiten – izdelala ga je oseba, ki so ji to delo naprtili. Naprtili je zelo primerna beseda, saj gre za zahtevno opravilo, ki izdelovalcu urnikov nemalokrat pusti ogromno sivih las. Ne bi se rad postavil v koo uboca, ki kona z izdelavo urnika, nakar ves zgroen ugotovi, da je spregledal prekrivanje dveh olskih ur, kar pomeni, da bi uenci morali biti pri dvoje urah hkrati. Tako mu seveda ne preostane drugega, kot da poskusi znova ter po nekaj neuspelih poskusih vendarle izdela urnik, na katerega pa vseeno leti kopica pripomb.

Napredneje rone metode izdelovanja urnikov potekajo s pomojo modelov in tabel, vendar tudi te pogosto ne vodijo do kakovostnih urnikov. Pri vseh je namre prisoten faktor omejenosti lovekih moganov, ki se po hitrosti obdelave podatkov niti priblino ne morejo kosati z dananjimi procesorji.

Prav zmogljivost procesorjev je tista, ki e od izuma raunalnika ene programerje k ustvarjanju optimizacijskih algoritmov za iskanje kakovostnejega urnika. Prav kakovost pa je lahko dvorezen me, saj nekateri urniki zadovoljijo potrebe uencev, a se premalo osredotoajo na potrebe uiteljev, in obratno. Kateri urnik je potemtakem najkakovostneji? Ali tak urnik sploh obstaja?

Znano je, da so za izdelavo najzahtevneji fakultetni urniki, zato od sedaj naprej obravnavamo le slednje. Seznam omejitev, ki jih moramo upotevati pri teh urnikih, je namre obseneji kot pri urnikih osnovnih in srednjih ol.

1.1 Vhodni podatki

e elimo izdelati urnik, v prvi vrsti potrebujemo podatke, ki jih moramo upotevati in kasneje tudi obdelati. Te podatke preberemo iz podatkovnih baz ali drugih vrst datotek, med katerimi vrh prestola zasedajo XML datoteke. Naslednji

primer ponazarja, kako je v XML datoteki predstavljen profesor Janez Novak, ki trikrat na teden predava predmet matematika.

```
<profesor>
   <ime>Janez</ime>
   <priimek>Novak</priimek>
   <predmet>
      <imePredmeta>matematika</imePredmeta>
      <steviloUrNaTeden>3</steviloUrNaTeden>
   </predmet>
</profesor>
```

Hierarhija znak omogoa enostavno vnaanje podatkov, hkrati pa poskrbi za preglednost. e enostavneje je dostopanje do posameznih podatkov, saj veina programskih jezikov e vsebuje metode za delo z XML datotekami.

Pri objektno orientiranih programskih jezikih (Delphi, C++, Java) vse vhodne podatke opiemo s posameznimi razredi. Poglejmo, katere razrede je smiselno implementirati ter katere podatke naj vsebujejo.

- Razred *Profesor* vsebuje osebne podatke profesorja (ime in priimek) ter podatke o predmetih, ki jih predava. Hkrati moramo za vsako uro v tednu vedeti, e je profesor takrat e zaseden, da mu sluajno ne bi dodelili dveh predavanj v istem asovnem terminu. To je najlaje preverjati s pomojo tabele zasedenosti, ki je podrobneje opisana na koncu poglavja.

- Razred *Skupina* predstavlja skupino tudentov. Kot podatke torej potrebujemo smer skupine (npr. pedagoka matematika) ter tevilo tudentov v njej. Podobno kot pri profesorjih, tudi tukaj potrebujemo tabelo zasedenosti.

- Razred *Predavalnica* nosi podatke o kapaciteti predavalnice, njeno oznako ter podatke o zasedenosti, ki so predstavljeni s tabelo.

- Razred *Predmet* vsebuje podatke o posameznih predmetih. Kot podatke tega objekta je smiselno navesti ime predmeta ter tedensko tevilo ur njegove izvedbe. Zanima nas tudi, katerim skupinam je predmet namenjen, ter v katerem semestru se bo predaval.

Kar pri treh razredih je kot podatek navedena tabela zasedenosti, zato si jo podrobneje oglejmo. Vsako tabelo doloata njena velikost ter tip vrednosti. Naslednji izraun nam poda velikost tabele:

$$\text{velikostTabele} = \text{teviloUrNaDan} \cdot \text{teviloDelovnihDni}$$

e bi predpostavili, da tudijski dan traja od sedme ure zjutraj do osmih zveer in da imamo pet delovnih dni, je velikost nae tabele 65 polj. Velikost te tabele torej pove, koliko ur na teden obravnavamo. Ker se olski tedni periodino ponavljajo, je seveda dovolj, e podamo zasedenost le za en teden.

Tip vrednosti tabele ni striktno doloen. Tako bi nekateri programerji uporabili celotevilsko tabelo, kjer bi vrednost 1 pomenila, da je oseba zasedena, vrednost 0 pa bi predstavljala njeno dosegljivost. Podobno bi postopali, e bi izbrali logini tip (npr. `bool` v C++), kjer bi vrednost `true` predstavljala dosegljivost, `false` pa zasedenost.

1.2 Omejitve

Programa za izdelavo (nekakovostnega) urnika naeloma ni teko narediti. Potrebujemo le metodo, ki nakljuno prireja termine posameznim predmetom. Teava pri taknem postopanju je, da pri nakljunem vstavljanju nismo upotevali nobenih omejitev, prav zadoanje omejitvam pa je tisto, kar odlikuje dober urnik.

1.2.1 Stroge omejitve

Verjetno e niste videli profesorja, ki vsakih pet minut tee iz ene predavalnice v drugo, ker so mu izdelalovalci urnika dodelili dve predavanji v istem asovnem terminu. Razlog je preprost – vsak urnik mora biti izdelan tako, da zadoa vsem strogim omejitvam. Poglejmo si nekaj omejitev, ki jih urnik ne sme kriti:

- V urnik moramo razdeliti vse ure vseh predmetov.
- Noben profesor ne sme imeti ve predavanj hkrati.
- Nobena skupina ne sme imeti ve predavanj hkrati.
- Nobena predavalnica ne sme biti dodeljena veim predavanjem hkrati.
- Predavalnica mora imeti zadostno kapaciteto.
- Predavanja posebne vrste (npr. vaje iz raunalnitva) se morajo izvajati v ustrezni predavalnici.
- Urnik mora zadoati zastavljenemu asovnemu okvirju (npr. od sedmih zjutraj do osmih zveer).

Urnik, ki zadoa vsem strogim omejitvam, imenujemo zadosten urnik. Zadostni urnik tako ne kri nobenega izmed zgoraj natetih nael, a e zdale ne ustreza merilom kakovostnega urnika.

Algoritem je najbolje implementirati tako, da do krenja strogih omejitev sploh ne more priti. Tabela zasedenosti je dober primer pametne implementacije, saj tako e v osnovi prepreujemo dodeljevanje t.i. prekrivanj. e eli algoritem dodeliti predavanje doloeni predavalnici, mora najprej v tabeli zasedenosti preveriti, e je v tistem terminu le-ta e na voljo, sicer se vstavljanje ne izvede.

1.2.2 ibke omejitve

V splonem ne zahtevamo, da urnik zadoa vsem ibkim omejitvam, ker je to praktino neizvedljivo. Resda pa so prav ibke omejitve tiste, ki doloajo kakovost urnika, in dober algoritem bo poskrbel, da se bo zadostilo veini le-teh. Natejmo nekaj izmed njih:

- Urnik naj vsebuje im manj prostih ur za profesorje in tudente.
- tudentje naj se im manj selijo iz predavalnic.
- Predmet z veliko urami naj bo enakomerno porazdeljen ez celotno periodo (teden).
- Predavanja naj se izvajajo ob urah, ki so najugodneje za pouevanje.

- V petek naj se tudentom dodeli im manj predavanj.
- tudentom se naj v asu kosila dodeli prosta ura.

Obstaja e mnogo drugih ibkih omejitev, ki se jim je smiselno posvetiti. Nikomur najbr ne bi bilo preve ve, e bi uram portne vzgoje sledila e vrsta predavanj, saj bi tako prepoteni in izrpani tudentje teko sledili tempu pouka.

Pri ronem sestavljanju urnika se ponavadi sestavi zadostni urnik, ki zadoa le najpomembnejim ibkim omejitvam. Dobro zasnovan algoritem pa lahko preverja veliko tevilo ibkih omejitev, zato snovalci programov prav temu posveajo ogromno pozornosti, saj bo njihov konni produkt le tako zadovoljil profesorje in tudente.

1.3 Merjenje kakovosti

Kakovost urnika se meri z zadoanjem ibkim omejitvam, pri emer e predpostavljamo zadostnost urnika (zadoanje strogim omejitvam). Ta nain je smiseln predvsem zato, ker si urnik, ki ne zadoa strogim omejitvam, tega naziva sploh ne zaslui.

lovek, ki najprej ustreli in ele nato razmisli, bi verjetno dejal, da tevilo zadoenih ibkih omejitev doloa kakovost urnika, kar pa je dale od resnice. Medtem ko so stroge omejitve po pomembnosti (skoraj) enakovredne, se ibke omejitve med seboj krepko razlikujejo. Tako je bolj pomembno, da urnik vsebuje manj prostih ur za tudente in profesorje, kot pa je dodeljena prosta ura v asu kosila. Kako bi torej ovrednotili posamezne ibke omejitve?

V splonem uporabljamo standardni postopek uteevanja omejitev, pri emer vsaki ibki omejitvi dodelimo celotevilsko ute glede na njeno pomembnost. Vpraanje je tudi, kako doloiti to pomembnost. e programer doloa utei po lastni presoji, so zagotovo zelo subjektivno dodeljene. Dosti bolj smiselno je anketiranje, kjer bi tudentje in profesorji razlinih izobraevalni ustanov ovrednotili posamezne ibke omejitve z ocenami od 1 do 10. S tem bi nastavljene utei pridobile na objektivnosti, verjetnost, da bo urnik dobro sprejet s strani veine, pa bi se poveala.

Kot zanimivost naj omenimo, da se nekateri programerji odloajo tudi za uteitev strogih omejitev s to razliko, da so vrednosti teh utei izjemno velike, zato se bo algoritem (skoraj) vedno »odloil«, da jih ne bo prekril. Tak nain uteevanja pride v potev takrat, ko algoritem v osnovi ne zadosti strogim omejitvam urnika.

Ko enkrat ustrezno nastavimo vrednosti utei, je treba ugotoviti, kako bomo postopali v nadaljevanju. Algoritem za izdelavo urnikov je najbolje zasnovati tako, da po vsaki spremembi izmeri njegovo kakovost. Tako moramo na vsakem koraku ugotavljati, katere omejitve so bile krene ter kolikokrat se kritev pojavi. S k_n oznaimo tevilo kritev omejitve n, z u_n pa njeno ute. Z naslednjo formulo izraunamo kakovost urnika:

$$k_1 \cdot u_1 + k_2 \cdot u_2 + \cdots + k_{n-1} \cdot u_{n-1} + k_n \cdot u_n$$

Nija vrednost formule pomeni kakovostneji urnik, izraunamo pa jo na vsakem koraku. Korak pri izdelavi urnika predstavlja eno izmed operacij nad njim (oznauje njegovo spremembo).

V 2. poglavju bomo opisali postopek gradnje urnika, saj bomo podrobneje spoznali operacije nad urnikom in ve vrst optimizacijskih metod, ki jih lahko

brez oklevanja poimenujemo kar osrje programa za avtomatizirano izdelavo urnika.

2 Avtomatizirana izdelava urnika

V 1. poglavju smo izvedeli, da rona izdela urnikov ni tako preprosta, kot bi si na prvi pogled predstavljali. e veji izziv predstavlja implementacija algoritma, ki da (fakultetni) urnik. Obstaja pa velika razlika med prvim in drugim pristopom, saj ko je algoritem enkrat dobro napisan, smo se teav z izdelovanjem urnika za vselej reili. Tako nas vsako leto aka le spreminjanje vhodnih podatkov, ki jih algoritem sprejme in nato iz njih izdela celovit urnik. Te vhodne podatke smo si skupaj s strogimi in ibkimi omejitvami e podrobneje ogledali v prvem delu, tako da se lahko tokrat osredotoimo na osrje programa.

Za lajo ilustracijo bi lahko potegnili kar nekaj vzporednic med programiranjem in – ne boste verjeli – kuhanjem. Za kuhanje potrebujemo recept, ki je sestavljen iz dveh delov. Prvi del predstavljajo sestavine, ki jih potrebujemo za kuhanje, glavni del pa predstavlja postopek, kako iz teh sestavin pripraviti okusno kosilo. Postopki kuhe se med seboj razlikujejo in nekateri pripeljejo do nekoliko slastnejega obroka kot drugi.

e se ob upotevanju zgornjih opomb vrnemo k programiranju, sestavine v naem programu e imamo, to so namre podatki o profesorjih, predavalnicah, skupinah ter predmetih. Prav tako imamo izbrane utei za posamezne omejitve. Manjka le e glavni del recepta, ki podatke ustrezno obdela ter iz njih sestavi urnik. Obstaja ve postopkov oziroma metod, ki vodijo do konnega urnika, podobno kot pri kuhi pa je tudi tukaj kakovost konnega izdelka odvisna od izbire najustrezneje metode.

Konni produkt, ki ga bomo ustvarili s pomojo kasneje omenjenih metod, bo urnik, ki je predstavljen z mnoico *sreanj*. Posamezno sreanje je sestavljeno iz predmeta, profesorja, ki predmet predava, ter skupin, ki jim je predmet namenjen. Sreanja so nato vstavljena v termine predavalnic, tako da se za vsako sreanje ve, ob kateri uri in v kateri predavalnici se bo izvajalo.

2.1 Porena metoda

Porena metoda je ena izmed strategij, s katerimi lahko reujemo optimizacijske probleme – to pomeni, da za dani problem iemo najboljo (optimalno) reitev. Metoda na vsakem koraku izmed vseh monih reitev izbere tisto, ki daje (trenutno) najveji dobiek oziroma najbolj povea vrednost kriterijske funkcije.

Podobno kot porena metoda nemalokrat postopamo tudi sami. Boini nakupi v trgovine prinaajo dolge akalne vrste in ponavadi se odloimo, da bomo z vozikom stopili do blagajne, na kateri aka najmanj ljudi. ele po polurnem poljem priblievanju blagajni ugotovimo, da bi pri veini drugih blagajn prili na vrsto prej, saj so tam ljudje opravljali le manje nakupe.

Znano je, da porena metoda pri nekaterih problemih odpove, je pa vseeno zelo uporabna tehnika za iskanje reitev, ko ne zahtevamo njihove optimalnosti. Urnik, ki ga dobimo s pomojo porene metode, je lahko namre e vedno za nekaj nivojev bolji od rono izdelanega. Naslednje vrstice bodo pojasnile, kako implementirati postopek porene metode za primer urnika.

```
(1) Sprehod po vseh predmetih.
```

```
{
   (2) Sprehod po vseh predavalnicah.
   {
      (3) Raunanje cene za proste termine.
      (4) Primerjava cene z najboljo.
   }
   (5) Vstavljanje sreanja v urnik.
}
```

Zgoraj predstavljena shema je le grob opis postopka, ki ga nad urnikom izvaja porena metoda, zato si je smiselno podrobneje ogledati posamezne korake.

1. Z zanko se sprehodimo po vektorju vseh predmetov. Predmet na i-tem koraku oznaimo s predmet_i.

2. Naslednja vgnezdena zanka nas popelje po vseh predavalnicah. Predavalnico na j-tem koraku oznaimo s predavalnica_j.

3. Za vse proste termine predavalnice_j izraunamo ceno vstavljanja sreanja_i, ki ga sestavlja predmet_i, profesor, ki ta predmet predava, ter skupina, kateri je predmet namenjen.

4. e je trenutna cena bolja od najbolje dosedanje, potem najbolja dosedanja dobi vrednost trenutne. Hkrati si zapomnimo, za kateri termin smo izraunali ceno vstavljanja, da bomo predmet_i na koncu vstavili v termin z najboljo ceno vstavljanja.

5. Ko preverimo vse proste termine vseh predavalnic, vstavimo sreanje_i v termin z najboljo ceno.

6. Sedaj se ponovno vrnemo na zaetek in pogledamo najugodneji termin za naslednji predmet. Postopek ponavljamo, dokler se zunanja zanka, ki se sprehaja po vseh predmetih, ne zakljui. S tem smo namre v urnik vstavili vse predmete.

2.2 Optimizacijske metode

Optimizacijske metode se e desetletja uporabljajo za reevanje optimizacijskih problemov. Nekatere manj, druge bolj uspeno reujejo zapletene probleme, pri emer je eksponentno naraajoa zmogljivost procesorjev le voda na njihov mlin, saj tako lahko v krajem asu izvedejo ve operacij. Konne reitve, ki jih dobimo z optimizacijskimi metodami, so lokalni ali globalni optimumi.

e elimo spoznati optimizacijske metode, se moramo najprej seznaniti z dvema pojmoma. *Operacija* je vsaka sprememba oziroma poseg v urnik med izvajanjem optimizacije. Na primeru urnika si oglejmo, katere so smiselne operacije:

- *Vstavljanje* sreanja v urnik.
- *Brisanje* sreanja iz urnika.
- *Zamenjava* dveh sreanj v urnik.

Okolica operacije je mnoica vseh monih sprememb, ki jih lahko s to operacijo opravimo na nekem koraku optimizacijskega postopka. Okolica zamenjave dveh sreanj v urniku je torej mnoica vseh monih zamenjav sreanj v trenutnem urniku.

Sistem okolic je mnoica vseh monih sprememb, ki jih lahko z vsemi operacijami iz okolic sistema opravimo na nekem koraku optimizacijskega postopka. Oboroeni z novimi pojmi se podajmo v naslednje vrstice.

2.2.1 Lokalno vzpenjanje

Lokalno vzpenjanje temelji na opazkah, da izbira irega sistema okolic prinaa kakovostneje reitve. Sistem okolic v primeru urnika predstavljajo prej omenjene operacije vstavljanja, brisanja ter zamenjave sreanj. Osnovno lokalno vzpenjanje se teoretino sploh ne razlikuje od porene metode, saj tudi pri tem postopku iz okolice vzamemo najboljo reitev. Kaj pa, e bi lokalno vzpenjanje nadgradili?

Do sedaj smo vedno dopuali le izbiro najbolje mone reitve iz doloene okolice, kar nas je hitro pripeljalo do lokalnega maksimuma, iz katerega pa se je bilo nemogoe izviti, saj je najbolja reitev iz okolice lokalnega maksimuma ponavadi kar lokalni maksimum sam. Nadgradnja lokalnega vzpenjanja je jasno razvidna iz naslednje sheme.

```
(1) Izvajaj dokler...
{
   (2) Iz sistema okolic izberi okolico.
   (3) Iz okolice izberi spremembo.
   (4) Izvedi spremembo.
}
```

Nadobudneem priporoamo, da najprej sami razmislijo, kaj je globja vsebina posameznih korakov, in si ele nato ogledajo njihov natanen opis.

1. Zunanjo zanko izvajamo, dokler ne doseemo enega izmed ustavitvenih pogojev. e nekaj asa kljub spreminjanju operacij nad urnikom vedno znova obstojimo na isti reitvi, potem to najverjetneje pomeni, da smo prili do lokalnega ekstrema. Drugi ustavitveni pogoj je zadovoljivo velika vrednost kriterijske funkcije.

2. Nakljuno izberemo okolico trenutne reitve. Z okolico izberemo tudi operacijo, ki jo izvajamo na tem koraku (npr. zamenjava dveh sreanj).

3. e v tem koraku izberemo tisto zamenjavo sreanj, ki izmed vseh monih najbolj povea vrednost kriterijske funkcije, potem postopamo enako, kot porena metoda. e elimo nadgraditi postopek lokalnega vzpenjanja, ne potrebujemo nujno najbolje mone zamenjave, ampak prvo, ki povea vrednost kriterijske funkcije. Izjemoma dovolimo tudi tisto zamenjavo, ki vrednost kriterijske funkcije nekoliko zmanja, kar se na prvi pogled zdi nesmiselno, nas pa dolgorono pripelje do boljih reitev.

4. Izbrano zamenjavo dveh sreanj izvedemo na trenutnem urniku. Vidimo, da en korak lokalnega vzpenjanja res predstavlja eno spremembo nad urnikom, v konkretnem primeru smo izvedli (eno) zamenjavo dveh sreanj.

Na tak nain izboljana metoda lokalnega vzpenjanja je asovno veliko zahtevneja od porene metode, saj se vrednost kriterijske funkcije ne poveuje tako drastino. Vseeno pa vidimo, da se poasi pribliujemo lokalnemu maksimumu in sasoma nas bo postopek pripeljal do njega. Nakljuna izbira okolic ter sprejemanje ne nujno najbolje reitve nas varujeta, da ne bomo obstali v prvem lokalnem maksimumu, ampak bomo vedno znova iskali bolje reitve.

Slabost do sedaj obravnavanih tehnik e vedno ostaja – ko enkrat obtiimo v lokalnem maksimumu, se iz njega ne moremo reiti. Naslednja metoda poskua na svojevrsten, a zelo sistematien nain odpraviti to teavo.

2.2.2 Iskanje s tabuji

e samo ime spominja na prepovedi, ki jih le-ta skriva v svojem naroju. Kaj sploh prepovedujemo? Glede na to, da bi naj iskanje s tabuji odpravilo problem vraanja v isti lokalni maksimum, prepovedujemo izvajanje sprememb, s katerimi bi se vrnili v e najden lokalni maksimum.

Reitev se ponuja kar sama – za naslednjih$n$ korakov prepovemo vsako spremembo, ki jo opravimo v koraku optimizacije. Izbira konstante n je odvisna od samega problema, saj ne obstaja neki sploen n, ki bi nas v vsakem primeru pripeljal iz lokalnega ekstrema. Imamo torej seznam prepovedanih sprememb – *tabujev* in na vsakem koraku za izbrano spremembo preverimo, e je element seznama tabujev. e spremembe ni v seznamu tabujev, jo preprosto izvedemo, v primeru, da je, pa postopek vrnemo na iskanje druge spremembe. Algoritem iskanja s tabuji se od lokalnega vzpenjanja torej razlikuje le v tem, da pred izbiro spremembe preveri, e je vsebovana v seznamu prepovedanih sprememb. Poglejmo, zakaj je takno postopanje smiselno.

Naj bo $n = 100$ ter s_1 sprememba urnika, ki nas je pripeljala v lokalni maksimum. To spremembo (npr. zamenjava sreanja$_i$ in sreanja$_j$) sedaj shranimo v seznam tabujev. V naslednjem koraku izvedemo spremembo s_2, ki se prav tako shrani v seznam prepovedanih sprememb. Podobno naredimo za vse spremembe v nadaljnih korakih. S tem poskrbimo, da se lahko v isti lokalni maksimum vrnemo ele ez $n = 100$ korakov, saj se sprememba s_1, ki vodi do lokalnega maksimuma, ele tedaj izbrie iz seznama prepovedanih sprememb.

e smo dosegli lokalni maksimum in vrednost kriterijske funkcije v n korakih uspemo reiti iz okolice tega lokalnega maksimuma, potem se je metoda iskanja s tabuji izkazala kot uspena.

2.3 Konni algoritem

Za konec si oglejmo, kako je videti optimizacijski del algoritma za iskanje najboljega urnika, pri emer je najbolji urnik tisti, ki s krenjem omejitev pridobi najmanj negativnih tok, oziroma ki ima najvejo vrednost kriterijske funkcije.

```
(1) r = reitev, dobljena s poreno metodo.
(2) Izvajaj, dokler...
{
   (3) Z optimizacijo izboljaj r.
   (4) Oceni r.
}
```

1. V prvem koraku izvedemo poreno metodo, ki reitev urnika pripelje v lokalni maksimum.

2. Zanko izvajamo, dokler ne doseemo enega izmed ustavitvenih pogojev.

3. Reitev r, ki smo jo dobili s pomojo porene metode, izboljamo z lokalnim vzpenjanjem ali iskanjem s tabuji. Kot je e bilo povedano, je iskanje s tabuji nekoliko primerneja metoda, saj nas bo laje pripeljala iz lokalnega maksimuma, do katerega smo prili s poreno metodo. Vsekakor pa ni ni narobe, e sreo poskusimo tudi z lokalnim vzpenjanjem.

4. Na vsakem koraku ocenimo izboljamo reitev r. e je vrednost kriterijske funkcije za trenutno reitev prela zastavljeno zadovoljivo mejo, potem se program zakljui. Prav tako se zakljui, e po n korakih e vedno tiimo v istem lokalnem maksimumu.

To je le ena izmed monosti, kako zastaviti optimizacijski potek, s katero smo eleli predstaviti simbiozo opisanih metod. Ni ne bi bilo narobe, e bi za iskanje najboljega urnika posamino uporabili katero izmed tehnik – mogoe bi katero izmed teh postopanj dalo celo bolji rezultat.

2.4 Zakljuek

Sledili smo receptu in s tem izdelali »okusen obrok«, treba ga je le e ustrezno postrei gostom za mizo. S tem imamo v mislih grafini uporabniki vmesnik (Graphical User Interface), ki mora urnik uporabnikom predstaviti na razumljiv in pregleden nain. Konnega izgleda izdelka nikakor ne smemo zanemariti, saj je ta med uporabniki vreden skoraj toliko kot kakovosten urnik. Odloili smo se, da za razmiljanje o tej temi vai domiljiji pustimo prosto pot.

Spoznali smo torej tri metode za reevanje optimizacijskih problemov, pri emer pa niti za eno ne moremo z gotovostjo trditi, da nas bo pripeljala do globalnega maksimuma. To nekako opozarja na kompleksnost problemov, hkrati pa tudi na neraziskanost tega podroja. Verjamemo, da bo prihodnost prinesla postopke, ki bodo s pomojo tedanjih ve tiso-jedrnih procesorjev znali poiskati globalne reitve za nekatere dananje optimizacijske probleme. Prihodnost pa bo hkrati prinesla tudi zahtevneje probleme in s tem nove izzive za velike ume tistega asa.